Лабораторна робота №4. Пакування застосунку в Helm-чарт

Дисципліна: «Технології DevOps» · Студент: MarcoGr

Репозиторій: https://github.com/MarcoGr11/DevOps

Маніфести лаби №3 перенесено в Helm-чарт `chart/` (booking 0.1.0, appVersion 1.0.0): образи, репліки, ресурси, ConfigMap, Secret та Ingress-хост винесено у `values.yaml`, окремі значення для середовищ — у `values-dev.yaml` і `values-prod.yaml`. До чарта додано pre-upgrade hook (Job перевірки БД) і `helm test`. Середовище: minikube, Kubernetes v1.37.0, Helm v3.22.0; реліз `booking` у namespace `coworking-booking`.

6. Два середовища

6.a Різниця dev і prod без встановлення

Рендеримо чарт для dev і prod командою `helm template`, нічого не встановлюючи, і порівнюємо результат.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm template booking chart/ -f chart/values-dev.yaml > /tmp/dev.yaml
helm template booking chart/ -f chart/values-prod.yaml \
  --set secret.postgresPassword=x \
  --set secret.djangoSecretKey=y > /tmp/prod.yaml
diff /tmp/dev.yaml /tmp/prod.yaml | grep -v -E "POSTGRES_PASSWORD|DJANGO_SECRET_KEY"
16,17c16,17
---
36c36
<   DJANGO_DEBUG: "1"
---
>   DJANGO_DEBUG: "0"
38c38
<   BOOKING_TIMEOUT_CHECK_INTERVAL: "15"
---
>   BOOKING_TIMEOUT_CHECK_INTERVAL: "60"
60c60
<       storage: 1Gi
---
>       storage: 10Gi
122c122
<   replicas: 1
---
>   replicas: 3
145,146c145,146
<         checksum/config: 6f54ed050f6387191176eccec49e00ce3f888e67ccb8d3f0059914230e3be4b2
<         checksum/secret: 355483a988e7fdc8208b234448966c1a913b1aa1bb0d6fdd6de27952edd3a594
---
>         checksum/config: b53b8ce2a13f4596464808ffa1afde5a85938a061151c2e7245891ec80ef4f30
>         checksum/secret: 4a5e09d4ec4e5b4700cc9855de58101211a93906ea146b41dad59c794c38eaa4
150c150
<           image: ghcr.io/marcogr11/devops-api:1.0.0
---
>           image: ghcr.io/marcogr11/devops-api:1.1.0
162,163c162,163
<               cpu: 250m
<               memory: 192Mi
```

```
---
>               cpu: "1"
>               memory: 512Mi
165,166c165,166
<               cpu: 50m
<               memory: 96Mi
---
>               cpu: 250m
>               memory: 256Mi
243,244c243,244
<               cpu: 250m
<               memory: 256Mi
---
>         checksum/config: b53b8ce2a13f4596464808ffa1afde5a85938a061151c2e7245891ec80ef4f30
>         checksum/secret: 4a5e09d4ec4e5b4700cc9855de58101211a93906ea146b41dad59c794c38eaa4
292c292
<           image: ghcr.io/marcogr11/devops-worker:1.0.0
---
>           image: ghcr.io/marcogr11/devops-worker:1.1.0
301,302c301,302
<               cpu: 100m
<               memory: 96Mi
---
>               cpu: 250m
>               memory: 256Mi
304,305c304,305
<               cpu: 20m
<               memory: 32Mi
---
>               cpu: 100m
>               memory: 128Mi
323c323
<     - host: "dev.booking.local"
---
>     - host: "booking.example.com"
```

Рис. 1. Різниця маніфестів dev і prod (helm template + diff)

Обидва рендери пройшли без помилок, а `diff` показує лише ті поля, які перевизначає `values-prod.yaml`: `DJANGO_DEBUG` "1" → "0", `BOOKING_TIMEOUT_CHECK_INTERVAL` "15" → "60", розмір PVC `storage: 1Gi` → `10Gi`, `replicas: 1` → `3` для api, образи `devops-api:1.0.0` → `1.1.0` і `devops-worker:1.0.0` → `1.1.0`, більші requests/limits (api `cpu: "1"`, `memory: 512Mi`) та Ingress-хост `dev.booking.local` → `booking.example.com`. Змінюються й анотації `checksum/config` та `checksum/secret`, тому поди перезапускаються при зміні ConfigMap чи Secret. Рядки з паролями відфільтровано `grep -v`, від них лишився тільки заголовок `16,17c16,17`. Вивід не вмістився в одне вікно, тому склеєно два знімки; пунктир позначає кілька пропущених рядків між ними (ресурси postgres і початок блоку worker).

6.b Пріоритет values < -f < --set

Рендеримо Deployment api трьома способами, щоб побачити, яке значення перемагає.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % echo "--- values.yaml"
helm template booking chart/ -s templates/deployment-api.yaml \
  | grep -E "^  replicas:|image:"
echo "--- -f values-prod.yaml"
helm template booking chart/ -s templates/deployment-api.yaml \
  -f chart/values-prod.yaml \
  --set secret.postgresPassword=x --set secret.djangoSecretKey=y \
  | grep -E "^  replicas:|image:"
echo "--- -f values-prod.yaml --set"
helm template booking chart/ -s templates/deployment-api.yaml \
  -f chart/values-prod.yaml \
  --set secret.postgresPassword=x --set secret.djangoSecretKey=y \
  --set replicaCount.api=5 --set image.api.tag=1.0.0 \
  | grep -E "^  replicas:|image:"
--- values.yaml
  replicas: 2
          image: ghcr.io/marcogr11/devops-api:1.0.0
--- -f values-prod.yaml
  replicas: 3
          image: ghcr.io/marcogr11/devops-api:1.1.0
--- -f values-prod.yaml --set
  replicas: 5
          image: ghcr.io/marcogr11/devops-api:1.0.0
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 2. Перекриття значень: values.yaml < -f < --set

Той самий шаблон Deployment api відрендерено тричі. З самим `values.yaml` маємо `replicas: 2` і образ `1.0.0`. Із `-f values-prod.yaml` отримуємо `replicas: 3` і `1.1.0`, тобто файл перекрив значення за замовчуванням. Якщо додати `--set replicaCount.api=5 --set image.api.tag=1.0.0`, буде

`replicas: 5` і `1.0.0`: `--set` має найвищий пріоритет і перекриває і файл, і `values.yaml`.

7. Перевірка і встановлення

7.a helm lint і структура чарта

Перевіряємо чарт лінтером і показуємо його структуру.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm lint chart/
helm lint chart/ -f chart/values-dev.yaml
find chart -type f ! -name DEMO.md | sort
==> Linting chart/
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
==> Linting chart/
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
chart/Chart.yaml
chart/templates/NOTES.txt
chart/templates/_helpers.tpl
chart/templates/configmap.yaml
chart/templates/deployment-api.yaml
chart/templates/deployment-postgres.yaml
chart/templates/deployment-worker.yaml
chart/templates/hook-db-check.yaml
chart/templates/ingress-api.yaml
chart/templates/namespace.yaml
chart/templates/pvc.yaml
chart/templates/secret.yaml
chart/templates/service-api.yaml
chart/templates/service-postgres.yaml
chart/templates/tests/test-health.yaml
chart/values-dev.yaml
chart/values-prod.yaml
chart/values.yaml
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 3. helm lint і структура каталогу chart/

`helm lint` для чарта з базовими значеннями і з `values-dev.yaml` двічі показує `1 chart(s) linted, 0 chart(s) failed`; єдине повідомлення — інформаційне `icon is recommended`. Нижче видно структуру чарта: `Chart.yaml`, три файли values, шаблони всіх об'єктів лаби №3, `_helpers.tpl`, `NOTES.txt`, hook `hook-db-check.yaml` і тест `tests/test-health.yaml`.

7.b Набір об'єктів: чарт vs лаба №3

Порівнюємо, які об'єкти створює чарт і які були в маніфестах лаби №3.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % echo "--- helm template (dev):"
helm template booking chart/ -f chart/values-dev.yaml | grep "^kind:" | sort | uniq -c
echo "--- k8s/ (лаба 3):"
cat k8s/*.yaml | grep "^kind:" | sort | uniq -c
--- helm template (dev):
   1 kind: ConfigMap
   3 kind: Deployment
   1 kind: Ingress
   1 kind: Job
   1 kind: PersistentVolumeClaim
   1 kind: Pod
   1 kind: Secret
   2 kind: Service
--- k8s/ (лаба 3):
   1 kind: ConfigMap
   3 kind: Deployment
   1 kind: Ingress
   1 kind: Namespace
   1 kind: PersistentVolumeClaim
   2 kind: Service
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 4. Об'єкти чарта порівняно з маніфестами лаби №3

Чарт створює ті самі об'єкти, що й маніфести лаби №3: 1 ConfigMap, 3 Deployment, 1 Ingress, 1 PVC, 2 Service. До них додано Secret (у лабі №3 його створювали командою `kubectl create secret`), Job (pre-upgrade hook) і Pod (`helm test`). Окремого Namespace у рендері немає, бо namespace створює сам Helm через `--create-namespace`.

7.c helm install

Встановлюємо реліз `booking` з dev-значеннями.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get namespace coworking-booking
helm install booking chart/ -f chart/values-dev.yaml \
  -n coworking-booking --create-namespace
Error from server (NotFound): namespaces "coworking-booking" not found
NAME: booking
LAST DEPLOYED: Tue Oct 20 01:52:27 2026
NAMESPACE: coworking-booking
STATUS: deployed
REVISION: 1
NOTES:
Release "booking" (booking 0.1.0) installed in namespace "coworking-booking".
api image: ghcr.io/marcogr11/devops-api:1.0.0, replicas: 1

Wait for the pods:
  kubectl get pods -n coworking-booking -l app.kubernetes.io/instance=booking -w

Via Ingress (host dev.booking.local):
  minikube tunnel   # in a separate terminal; add "127.0.0.1 dev.booking.local" to /etc/hosts
  curl http://dev.booking.local/api/health/

Via port-forward:
  kubectl port-forward -n coworking-booking svc/booking-api 8080:80
  curl http://localhost:8080/api/health/

Smoke test:
  helm test booking -n coworking-booking
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 5. Встановлення релізу booking (dev) і NOTES

Перед встановленням `kubectl get namespace` повертає `NotFound`, тобто стан лаби №3 повністю видалено. `helm install` створює namespace і реліз: `STATUS: deployed`, `REVISION: 1`, `LAST DEPLOYED 01:52:27`. Далі виводяться NOTES із чарта: версія `booking 0.1.0`, образ `devops-api:1.0.0` з `replicas: 1` (dev), хост `dev.booking.local` і підказки для port-forward і `helm test`.

7.d list, status, get values, поди

Перевіряємо стан релізу, передані значення і поди.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl rollout status deployment/booking-api -n coworking-booking --timeout=180s
helm list -n coworking-booking
helm status booking -n coworking-booking | head -6
helm get values booking -n coworking-booking \
  | sed -E 's/(postgresPassword|djangoSecretKey): .*/\1: ********/'
kubectl get pods -n coworking-booking
deployment "booking-api" successfully rolled out
NAME    NAMESPACE           REVISION        UPDATED                                 STATUS          CHART           APP VERSION
booking coworking-booking       1               2026-10-20 01:52:27.227579 +0300 EEST   deployed        booking-0.1.0   1.0.0
NAME: booking
LAST DEPLOYED: Tue Oct 20 01:52:27 2026
NAMESPACE: coworking-booking
STATUS: deployed
REVISION: 1
NOTES:
USER-SUPPLIED VALUES:
config:
  bookingTimeoutCheckInterval: "15"
  djangoDebug: "1"
image:
  api:
    tag: 1.0.0
  worker:
    tag: 1.0.0
ingress:
  enabled: true
  host: dev.booking.local
postgres:
  persistence:
    size: 1Gi
```

```
replicaCount:
  api: 1
  worker: 1
resources:
  api:
    limits:
      cpu: 250m
      memory: 192Mi
    requests:
      cpu: 50m
      memory: 96Mi
  postgres:
    limits:
      cpu: 250m
      memory: 256Mi
    requests:
      cpu: 50m
      memory: 96Mi
  worker:
    limits:
      cpu: 100m
      memory: 96Mi
    requests:
      cpu: 20m
      memory: 32Mi
NAME                                READY   STATUS    RESTARTS      AGE
booking-api-cbc7c7556-6bk7d         1/1     Running   1 (41s ago)   44s
booking-postgres-65c7b87bf9-2xqtj   1/1     Running   0             44s
booking-worker-7695b9f9bb-kkvv6     1/1     Running   2 (40s ago)   44s
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 6. Стан релізу, значення користувача і поди

Rollout api завершився (`successfully rolled out`). `helm list` показує реліз `booking`: ревізія 1, `deployed`, chart `booking-0.1.0`, app version `1.0.0`. `helm get values` виводить лише значення, передані користувачем з `values-dev.yaml`: interval "15", debug "1", теги `1.0.0`, хост `dev.booking.local`, 1Gi, по одній репліці api і worker, зменшені ресурси; секретів серед них немає. Усі три поди (`booking-api-cbc7c7556-6bk7d`, postgres, worker) мають статус `Running 1/1`. Перезапуски api і worker відбулися, поки стартував postgres.

7.e Де Helm зберігає стан

Показуємо Secret, у якому Helm зберігає ревізію релізу.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get secrets -n coworking-booking | grep helm
kubectl get secret sh.helm.release.v1.booking.v1 -n coworking-booking \
  -o jsonpath='{.metadata.labels}'; echo
sh.helm.release.v1.booking.v1   helm.sh/release.v1   1      69s
{"modifiedAt":"1792450347","name":"booking","owner":"helm","status":"deployed","version":"1"}
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 7. Secret sh.helm.release.v1.booking.v1 зі станом релізу

Ревізію релізу Helm зберігає в кластері як Secret `sh.helm.release.v1.booking.v1` типу `helm.sh/release.v1`. За мітками `name=booking`, `owner=helm`, `status=deployed`, `version=1` Helm знаходить історію релізу; кожна наступна ревізія додає новий такий Secret.

8. Життєвий цикл релізу

8.a upgrade через --set

Оновлюємо лише тег образу api, решта dev-значень лишається.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm upgrade booking chart/ -n coworking-booking \
  --reuse-values --set image.api.tag=1.1.0 | head -6
kubectl rollout status deployment/booking-api -n coworking-booking
sleep 10
curl -s -H "Host: dev.booking.local" http://localhost:8081/api/health/; echo
Release "booking" has been upgraded. Happy Helming!
NAME: booking
LAST DEPLOYED: Tue Oct 20 01:53:57 2026
NAMESPACE: coworking-booking
STATUS: deployed
REVISION: 2
Waiting for deployment "booking-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "booking-api" rollout to finish: 1 old replicas are pending termination...
deployment "booking-api" successfully rolled out
{"status":"ok","version":"1.1.0","pod":"booking-api-674f7cf49-nv7xc"}
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 8. Upgrade тегу api до 1.1.0 через --set

Завдяки `--reuse-values` змінився лише тег образу api, решта dev-значень залишилась без змін. Upgrade створив `REVISION: 2`, rollout пройшов, а запит через Ingress (`Host: dev.booking.local`) повертає `"version":"1.1.0"` від нового пода `booking-api-674f7cf49-nv7xc`.

8.b upgrade з values-prod

Переводимо реліз на prod-значення; паролі беремо з поточного Secret, щоб postgres прийняв той самий пароль.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm upgrade booking chart/ -n coworking-booking -f chart/values-prod.yaml \
  --set secret.postgresPassword="$(kubectl get secret booking-secret -n coworking-booking \
      -o jsonpath={.data.POSTGRES_PASSWORD} | base64 -d)" \
  --set secret.djangoSecretKey="$(kubectl get secret booking-secret -n coworking-booking \
      -o jsonpath={.data.DJANGO_SECRET_KEY} | base64 -d)" \
  | head -6
kubectl rollout status deployment/booking-api -n coworking-booking --timeout=300s
kubectl get pods -n coworking-booking
Release "booking" has been upgraded. Happy Helming!
NAME: booking
LAST DEPLOYED: Tue Oct 20 01:54:52 2026
NAMESPACE: coworking-booking
STATUS: deployed
REVISION: 3
Waiting for deployment "booking-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "booking-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "booking-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "booking-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "booking-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "booking-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "booking-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "booking-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "booking-api" rollout to finish: 1 old replicas are pending termination...
deployment "booking-api" successfully rolled out
NAME                                READY   STATUS        RESTARTS      AGE
booking-api-674f7cf49-2skrk         1/1     Terminating   0             18s
booking-api-674f7cf49-lqc6t         1/1     Terminating   0             18s
booking-api-674f7cf49-nv7xc         1/1     Terminating   0             72s
booking-api-7489549547-4c4sv        1/1     Running       0             12s
booking-api-7489549547-82zg8        1/1     Running       0             18s
booking-api-7489549547-c9vlg        1/1     Running       0             6s
booking-postgres-69f4d4fbc7-x2frj   1/1     Running       0             18s
booking-worker-66dfbfd595-pm25q     1/1     Running       0             17s
booking-worker-66dfbfd595-z9fzl     1/1     Running       1 (16s ago)   18s
booking-worker-7695b9f9bb-z76pg     1/1     Terminating   0             18s
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 9. Upgrade релізу на values-prod.yaml

Реліз переведено на prod-значення: `REVISION: 3`, `LAST DEPLOYED 01:54:52`. Паролі передано через `$(kubectl get secret … | base64 -d)`, тому в терміналі їх значень немає. Rollout видно поетапно: 1, потім 2 з 3 нових реплік, далі очікування видалення старих. Після цього працюють 3 поди `booking-api-7489549547-*` і 2 поди `booking-worker-66dfbfd595-*`, а старі `674f7cf49-*` і `7695b9f9bb-*` перебувають у `Terminating`. Postgres перестворено з prod-ресурсами (`booking-postgres-69f4d4fbc7-x2frj`).

8.c Історія і значення ревізій

Дивимось історію ревізій і що саме змінювалось між ними.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm history booking -n coworking-booking
v() {
  helm get values booking -n coworking-booking --revision $1 \
    | sed -E 's/(postgresPassword|djangoSecretKey): .*/\1: ********/'
}
echo "=== ревізія 1 -> 2"
diff <(v 1) <(v 2)
echo "=== ревізія 2 -> 3"
diff <(v 2) <(v 3)
REVISION        UPDATED                         STATUS          CHART           APP VERSION     DESCRIPTION
1               Tue Oct 20 01:52:27 2026        superseded      booking-0.1.0   1.0.0           Install complete
2               Tue Oct 20 01:53:57 2026        superseded      booking-0.1.0   1.0.0           Upgrade complete
3               Tue Oct 20 01:54:52 2026        deployed        booking-0.1.0   1.0.0           Upgrade complete
=== ревізія 1 -> 2
7c7
<     tag: 1.0.0
---
>     tag: 1.1.0
=== ревізія 2 -> 3
3,4c3,4
<   bookingTimeoutCheckInterval: "15"
<   djangoDebug: "1"
---
>   bookingTimeoutCheckInterval: "60"
>   djangoDebug: "0"
9c9
<     tag: 1.0.0
---
>     tag: 1.1.0
12c12
<   host: dev.booking.local
---
>   host: booking.example.com
15c15
<     size: 1Gi
---
>     size: 10Gi
17,18c17,18
---------------------------------------------------------------------------------------
>   api: 3
>   worker: 2
22,23c22,23
<       cpu: 250m
<       memory: 192Mi
---
>       cpu: "1"
>       memory: 512Mi
25,28d24
<       cpu: 50m
<       memory: 96Mi
<   postgres:
<     limits:
30a27,30
>   postgres:
>     limits:
>       cpu: "1"
>       memory: 1Gi
32,33c32,33
<       cpu: 50m
<       memory: 96Mi
---
>       cpu: 250m
>       memory: 512Mi
36,37c36,37
<       cpu: 100m
<       memory: 96Mi
---
>       cpu: 250m
>       memory: 256Mi
39,40c39,43
<       cpu: 20m
<       memory: 32Mi
---
>       cpu: 100m
>       memory: 128Mi
> secret:
>   djangoSecretKey: ********
>   postgresPassword: ********
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 10. helm history і різниця значень між ревізіями

`helm history` показує три ревізії: 1 — `Install complete`, 2 і 3 — `Upgrade complete`; остання має статус `deployed`. Різниця 1 → 2 складається з одного рядка `tag: 1.0.0 → 1.1.0`, тобто з того, що передано через `--set`. Різниця 2 → 3 — це повний перехід на prod: interval "60", debug "0", хост `booking.example.com`, `size: 10Gi`, репліки api 3 і worker 2, prod-ресурси. Також додався блок `secret:`, значення якого замасковано (`********`). Пунктир позначає кілька пропущених рядків між двома знімками (старі `api: 1`, `worker: 1`).

8.d rollback

Відкочуємо реліз на ревізію 1 (dev, образ 1.0.0).

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm rollback booking 1 -n coworking-booking
helm history booking -n coworking-booking
kubectl rollout status deployment/booking-api -n coworking-booking
sleep 10
curl -s -H "Host: dev.booking.local" http://localhost:8081/api/health/; echo
Rollback was a success! Happy Helming!
REVISION        UPDATED                         STATUS          CHART           APP VERSION     DESCRIPTION
1               Tue Oct 20 01:52:27 2026        superseded      booking-0.1.0   1.0.0           Install complete
2               Tue Oct 20 01:53:57 2026        superseded      booking-0.1.0   1.0.0           Upgrade complete
3               Tue Oct 20 01:54:52 2026        superseded      booking-0.1.0   1.0.0           Upgrade complete
4               Tue Oct 20 01:56:01 2026        deployed        booking-0.1.0   1.0.0           Rollback to 1
Waiting for deployment "booking-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "booking-api" rollout to finish: 1 old replicas are pending termination...
deployment "booking-api" successfully rolled out
{"status":"ok","version":"1.0.0","pod":"booking-api-cbc7c7556-hfhng"}
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 11. Rollback на ревізію 1

`helm rollback booking 1` пройшов успішно (`Rollback was a success!`). В історії з'явилась нова ревізія 4 з описом `Rollback to 1`: Helm не видаляє ревізії, а додає нову з конфігурацією першої. Після rollout запит повертає `"version":"1.0.0"` від пода `booking-api-cbc7c7556-hfhng`; це той самий ReplicaSet `cbc7c7556`, що і після install.

8.e uninstall

Видаляємо реліз і перевіряємо, що в namespace нічого не лишилось.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm uninstall booking -n coworking-booking
sleep 15
kubectl get all -n coworking-booking
helm list -n coworking-booking
kubectl get secrets -n coworking-booking | grep helm
release "booking" uninstalled
NAME                               READY   STATUS        RESTARTS   AGE
pod/booking-api-cbc7c7556-hfhng   1/1     Terminating   0          52s
NAME    NAMESPACE       REVISION        UPDATED STATUS  CHART   APP VERSION
No resources found in coworking-booking namespace.
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 12. helm uninstall і порожній namespace

`release "booking" uninstalled`. Через 15 секунд у namespace лишився тільки один под api, який ще завершує роботу (`Terminating`); жодного Deployment, Service чи ReplicaSet уже немає. `helm list` порожній, а `kubectl get secrets` відповідає `No resources found`, тобто Secret-и з історією релізу Helm теж видалив.

9. Hook і helm test

9.a pre-upgrade hook

Показуємо, що перед upgrade запускався Job перевірки БД.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % grep -n "helm.sh/hook" chart/templates/hook-db-check.yaml
kubectl get events -n coworking-booking \
  --field-selector involvedObject.kind=Job --sort-by=.lastTimestamp
10:    helm.sh/hook: pre-upgrade
11:    helm.sh/hook-weight: "0"
12:    helm.sh/hook-delete-policy: before-hook-creation,hook-succeeded
LAST SEEN   TYPE     REASON             OBJECT                  MESSAGE
3m14s       Normal   SuccessfulCreate   job/booking-db-check   Created pod: booking-db-check-22hc4
3m11s       Normal   Completed          job/booking-db-check   Job completed
2m20s       Normal   SuccessfulCreate   job/booking-db-check   Created pod: booking-db-check-bp6x8
2m17s       Normal   Completed          job/booking-db-check   Job completed
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 13. pre-upgrade hook booking-db-check

У шаблоні hook позначено анотаціями `helm.sh/hook: pre-upgrade`, `hook-weight: "0"` і `hook-delete-policy: before-hook-creation,hook-succeeded`. У подіях namespace видно два

запуски Job `booking-db-check` (поди `22hc4` і `bp6x8`), кожен із `SuccessfulCreate` і `Completed`. За часом вони збігаються з двома upgrade (ревізії 2 і 3); на rollback hook не запускався. Сам Job після успіху видалено політикою `hook-succeeded`, тому в `kubectl get jobs` його вже немає. Знімок зроблено після uninstall (події namespace ще зберігались), а вивід `kubectl get jobs -w` не знято.

9.b helm test

Запускаємо smoke-тест, вбудований у чарт.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % helm test booking -n coworking-booking \
  | grep -v -E "^(NOTES|Release|api image|Wait|Via|Smoke|  |$)"
NAME: booking
LAST DEPLOYED: Tue Oct 20 01:58:27 2026
NAMESPACE: coworking-booking
STATUS: deployed
REVISION: 1
TEST SUITE:     booking-test-health
Last Started:   Tue Oct 20 01:59:33 2026
Last Completed: Tue Oct 20 01:59:34 2026
Phase:          Succeeded
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 14. helm test: smoke-тест /api/health/

`helm test` запустив тестовий под `booking-test-health`, і тест пройшов: `Phase: Succeeded`, тривалість близько секунди (01:59:33 → 01:59:34). Тест виконувався на релізі, встановленому повторно після uninstall (`REVISION: 1`, `LAST DEPLOYED 01:58:27`) з тими самими dev-значеннями. Рядки NOTES відфільтровано `grep -v`.

Висновок

Чарт рендериться для dev і prod без встановлення, значення перекриваються в порядку values.yaml < -f < --set, а реліз пройшов повний цикл: install, два upgrade, history, rollback, test і uninstall. Набір об'єктів у чарті той самий, що в лабі №3, але з kubectl кожен файл застосовувався окремо, значення для середовищ доводилось правити вручну, а історія велась лише для одного Deployment. Helm керує всім застосунком як одним релізом: кожна ревізія зберігається в Secret кластера, rollback повертає всі об'єкти разом, а uninstall прибирає їх однією командою.